Practical 8A

Aim: Create a PWA application with an HTML page that acts as the starting point of the application. The HTML page contains a link to manifest.json.

Step 1: Create a Web Project

1. Open Visual Studio 2026 and click Create a new project.

2. Select ASP.NET Core Web App (Model-View-Controller), then click Next.

Step 2: Configure the Project

1. Project Name: BasicPWA

2. Framework: .NET 10 or the latest available version.

3. Click Create.

Step 3: Enable Static Files

1. Open Program.cs.

2. Configure the application to serve the default HTML page and static files. Use the following minimal setup:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Note: If you keep the MVC template's existing middleware and routes, ensure UseDefaultFiles() and UseStaticFiles() are included before app.Run().

Step 4: Create the Starting HTML Page

1. Right-click wwwroot → Add → New Item → HTML Page.

2. Name the file index.html.

3. Enter the following code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Basic PWA</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="manifest" href="manifest.json">
</head>
<body>
    <h1>My Progressive Web Application</h1>
    <p>This is the starting page of PWA.</p>
</body>
</html>
```

4. Save the file.

Step 5: Link the Manifest File in HTML

The link to manifest.json is already included inside the <head> section of index.html:

```
<link rel="manifest" href="manifest.json">
```

Step 6: Create Icons Folder

1. Right-click wwwroot → Add → New Folder.

2. Name the folder icons.

3. Add the following PNG files to the icons folder:

   • icon1.png — 192 × 192 pixels

   • icon2.png — 512 × 512 pixels

Step 7: Create Manifest File

1. Right-click wwwroot → Add → New Item → JSON File.

2. Name the file manifest.json.

3. Enter the following JSON:

```
{
  "name": "Basic Progressive Web Application",
  "short_name": "BasicPWA",
  "description": "Simple Example",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "orientation": "portrait",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

4. Save the file.

Step 8: Run the Application

1. Run the project in Visual Studio.

2. Open the browser URL and navigate to /index.html if it does not open automatically.

3. Verify that the page displays “My Progressive Web Application” and “This is the starting page of PWA.”

4. Check the browser address bar for the PWA installation option. The app must be served through HTTPS or localhost, and valid icon files must be present.

Result

A basic Progressive Web Application was created with an HTML starting page linked to a manifest.json file.

Practical 8B

Aim: Create a simple Web API that returns student data in JSON format.

Step 1: Create Web API Project

1. Open Visual Studio 2026.

2. Click Create a new project.

3. Select ASP.NET Core Web API and click Next.

Step 2: Configure Project

1. Project Name: StudentAPI

2. Framework: .NET 10 or the latest available version.

3. Click Create.

Step 3: Create Student Model

1. Right-click the project → Add → New Folder.

2. Name the folder Models.

3. Right-click Models → Add → Class.

4. Name the class Student.cs and enter:

```
namespace StudentAPI.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; } = "";
        public string Course { get; set; } = "";
        public int Age { get; set; }
    }
}
```

5. Save the file.

Step 4: Create API Controller

1. Right-click the project → Add → New Folder and name it Controllers, if it does not already exist.

2. Right-click Controllers → Add → Controller.

3. Choose API Controller - Empty.

4. Name it StudentController.cs.

Step 5: Write API Code

Open StudentController.cs and enter the following code:

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;

namespace StudentAPI.Controllers
{
    [ApiController]
```

```
    [Route("api/[controller]")]
    public class StudentController : ControllerBase
    {
        [HttpGet]
        public List<Student> GetStudents()
        {
            return new List<Student>
            {
                new Student
                {
                    Id = 1,
                    Name = "Rahul",
                    Course = "BSc IT",
                    Age = 20
                },
                new Student
                {
                    Id = 2,
                    Name = "Sneha",
                    Course = "BCA",
                    Age = 21
                }
            };
        }
    }
}
```

Save the file. Ensure Program.cs contains the controller configuration:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();

app.Run();
```

If the project already contains Swagger/OpenAPI setup, you can keep it; ensure AddControllers() and MapControllers() are present.

Step 6: Run the Application

1. Run the StudentAPI project in Visual Studio.

2. Note the HTTPS localhost address shown in the browser or launch profile.

Step 7: Test the API

1. Open a browser and enter the project URL followed by /api/student.

   Example: https://localhost:xxxx/api/student (replace xxxx with the actual port number).

2. The browser should display the student data in JSON format, including Rahul and Sneha.

Result

A simple ASP.NET Core Web API was created and tested successfully. It returns student data in JSON format.